Trabajo Práctico · Unidad 5

Administración y seguridad de Food Store: roles, auditoría, simulacro de incidente y anonimización · Base de Datos II · Tecnicatura Universitaria en Programación · UTN

Alumno: Moran, Joaquín Leandro · Trabajo individual.
Repositorio: github.com/moranjoaquin487-lgtm/tpi-food-store, carpeta `trabajos-practicos/unidad5_seguridad`.
Base: `foodstore`, en PostgreSQL 18 en mi PC. Todo, incluido el simulacro, se ejecutó **solo contra mi base de desarrollo**. Ningún archivo tiene contraseñas: se ponen con `\password` y se leen de `pgpass.conf`.

Parte	Archivos	Evidencia
A · Roles y permisos	`sql/roles.sql`, `sql/01_auth_funciones.sql`, `docs/kiro_roles_spec/`	`evidencias/A_03_verificacion_permisos.txt`, `A_04_comprobacion_auditoria_ia.txt`
B · Auditoría	`sql/configurar_auditoria.sql`, `sql/auditoria_test.sql`	`docs/informe_auditoria.md`, `docs/log_auditoria_prueba.txt`
C · Simulacro de incidente	`sql/simulacro_incidente.sql`, `scripts/simulacro_incidente.sh`, `scripts/anonimizar_log.py`	`docs/informe_incidente.md`, `docs/log_simulacro_anonimizado.txt`, `evidencias/C_simulacro_verificacion.txt`
D · Anonimización	`sql/usuario_anon.sql`, `sql/equivalencia_agregados.sql`	`docs/verificacion_equivalencia.md`, `evidencias/D_comprobacion_consultas_ia.txt`
E · Bitácora	`BITACORA.md`	Historial de Git, un commit por parte

Parte A · Roles de mínimo privilegio

Rol	Tipo	Justificación (una oración)
`rol_app_lectura`	Grupo	La tienda web muestra el catálogo y los pedidos: lee categorías, productos y pedidos, y nada con datos personales.
`rol_app_escritura`	Grupo	La tienda web registra ventas: inserta pedidos y sus líneas y descuenta stock, sin poder cambiar precios.
`rol_soporte`	Grupo	La mesa de ayuda atiende a usuarios y clientes: los consulta sin ver contraseñas, corrige el celular o la baja, y resetea contraseñas solo con la función.
`rol_reportes`	Grupo	El área comercial mira resultados agregados: lee vistas de ventas y la tabla anonimizada de usuarios, nunca datos personales.
`rol_auditor`	Grupo (adicional)	Quien investiga un incidente lee los registros de auditoría, y ningún dato del negocio.
`app_web`	Login	Login de la aplicación: lectura + escritura de la tienda y la función de autenticación.
`admin_datos`	Login	Administrador de datos: todo sobre las tablas del esquema salvo modificar la auditoría, sin ser superusuario ni poder crear roles.
`usr_soporte`, `usr_reportes`, `usr_auditor`	Login (adicionales)	Logins personales de cada área, para que el log diga quién hizo qué.

Matriz de permisos (la que se compara contra la base)

Rol	Permisos
`rol_app_lectura`	`SELECT` en `categoria`, `producto`, `pedido`, `detalle_pedido` y `vista_productos_vigentes`
`rol_app_escritura`	`INSERT` en `pedido` y `detalle_pedido`; **`UPDATE (stock)` en `producto`** (permiso por columna: no puede cambiar el precio); `EXECUTE` en `sp_registrar_pedido` y `fn_total_pedido`
`rol_soporte`	`SELECT` en `cliente`; **`SELECT` por columna en `usuario`** (id, nombre, apellido, mail, celular, rol, eliminado, created_at: todas menos `contrasena`); `UPDATE (celular, eliminado)`; `EXECUTE` en `fn_resetear_password`

Rol	Permisos
`rol_reportes`	`SELECT` en `mv_facturacion_cat_mes`, `v_top5_categorias`, `vista_productos_vigentes`, `vista_detalle_pedido_producto` y `usuario_anon`
`rol_auditor`	`SELECT` en `auditoria_seguridad` y `auditoria_precio`
`app_web`	Miembro de `rol_app_lectura` y `rol_app_escritura`; `EXECUTE` en `fn_autenticar`. No lee `usuario`: autentica con la función
`admin_datos`	`SELECT`, `INSERT`, `UPDATE`, `DELETE` en las 22 relaciones del esquema; la auditoría, solo lectura

- **Funciones sensibles:** `fn_autenticar` y `fn_resetear_password` son `SECURITY DEFINER` con `search_path` fijo. Son lo único que toca `usuario.contrasena` (hash con `pgcrypto`), y a `PUBLIC` se le revoca el `EXECUTE` que tiene por defecto.
- **Privilegios por defecto:** `ALTER DEFAULT PRIVILEGES FOR ROLE postgres` revoca el `EXECUTE` de `PUBLIC` en las funciones nuevas. Va en la forma global: con `IN SCHEMA public` no tenía efecto, porque los privilegios por esquema solo se suman a los globales.
- **Script repetible:** `roles.sql` crea los roles si no existen y revoca todo antes de otorgar. Ningún rol es superusuario, ni puede crear roles o bases.
- **Spec en Kiro:** la spec de roles (`docs/kiro_roles_spec/`) la revisó Kiro. Acepté 6 de sus 7 cambios y corregí un dato que escribió mal.

Verificación (`sql/verificacion_permisos.sql`)

Control	Resultado
`\du` y `\drg` (membresías, PostgreSQL 16+)	En la evidencia
Privilegios **de más** contra la matriz	**0 filas**
Privilegios **de menos** contra la matriz	**0 filas**
Privilegios para `PUBLIC`	**0 filas**
Membresías de más o de menos	**0 filas**
`admin_datos`: relaciones sin DML completo / con extras	0 y 0, de 22
`admin_datos` sobre la auditoría	Lee: sí · inserta, modifica, borra: no

Prueba con `SET ROLE`	Resultado
`app_web` registra un pedido con `CALL`	Funciona
`app_web` cambia un precio	Permiso denegado
`app_web` lee clientes	Permiso denegado
`usr_soporte` lee mails de usuarios	Funciona (147)
`usr_soporte` lee la contraseña	Permiso denegado
`usr_soporte` resetea con la función	Funciona
`usr_reportes` lee el top 5 / pedidos de clientes	Funciona / denegado
`admin_datos` borra la auditoría / crea un rol	Denegado / denegado

A.4 · Auditoría de `roles.sql` con OpenCode

OpenCode (agente Plan, en mi PC) propuso 7 cambios con su SQL. Antes de decidir, comprobé cada uno en mi base dentro de `BEGIN … ROLLBACK` (`sql/comprobar_auditoria_ia.sql`).

#	Propuesta	Decisión	Por qué
1	`admin_datos` puede modificar y borrar la auditoría; propone `REVOKE SELECT (contrasena)`	**Corregido en parte**	En la prueba, `admin_datos` borró las 6 filas de `auditoria_seguridad`: ahora la tiene en solo lectura. El `REVOKE SELECT (contrasena)` no tiene efecto: siguió leyendo las 147 contraseñas.
2	Falta `usuario_anon` para `rol_reportes`	Descartado	Ya lo otorga `usuario_anon.sql`, que corre después.
3	`app_web` lee los pedidos de todos los clientes	Riesgo aceptado	Ve los 200 009 pedidos de 19 998 clientes. Corregirlo exige seguridad por fila o cambiar la app.
4	`GRANT USAGE, SELECT ON ALL SEQUENCES` a `rol_app_escritura`	**Descartado: más amplio de lo necesario**	Las claves son `GENERATED ALWAYS AS IDENTITY` y `app_web` insertó un pedido sin ese permiso. Habría abierto las 11 secuencias del esquema.
5	Soporte puede dar de baja cualquier cuenta	Aceptado, pendiente	Soporte dio de baja a 9 administradores. Queda una función que excluya las cuentas `ADMIN`.
6	`ALTER DEFAULT PRIVILEGES … REVOKE ALL ON TABLES/SEQUENCES FROM PUBLIC`	Descartado	Una tabla y una secuencia nuevas no le dan nada a `PUBLIC`: no cambia nada.

#	Propuesta	Decisión	Por qué
7	La lectura de `cliente` y `usuario` está bien	Sin cambios	—

Parte B · Auditoría

`sql/configurar_auditoria.sql`, como postgres, guarda los valores actuales y aplica con `ALTER SYSTEM` + `pg_reload_conf()`, sin reiniciar. Se reconecta antes de mostrar los valores nuevos, porque las opciones de conexión rigen para las sesiones nuevas.

Parámetro	Antes	Después	Para qué
`log_connections`	`off`	`on`	Quién se conecta, desde dónde y con qué método
`log_disconnections`	`off`	`on`	Cuándo termina la sesión y cuánto duró
`log_statement`	`none`	`mod`	DDL y modificaciones: `INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`, `COPY FROM`
`log_min_error_statement`	`error`	`error`	El texto de toda sentencia que falla
`log_line_prefix`	`%m [%p]`	`%m [%p] %q%u@%d %r app=%a`	Usuario, base, IP y puerto de origen, y aplicación
`logging_collector`	`on`	`on`	Ya venía activo en la instalación de Windows

Secuencia de prueba (`sql/auditoria_test.sql`, conectado como `app_web`): una lectura del catálogo, un `INSERT` de un pedido de prueba con su línea y un `UPDATE` del stock. Después, un intento de conexión con contraseña incorrecta. El fragmento de log está en `docs/log_auditoria_prueba.txt`.

B.3 · Qué responde el log y qué queda afuera

Pregunta	Dónde aparece en el log
¿Quién?	`%u@%d` e `identity=`: `app_web@foodstore`, `method=scram-sha-256`
¿Qué?	`statement:` el texto del `INSERT` y del `UPDATE`
¿Cuándo?	`%m`: hora con milisegundos
¿Desde dónde?	`%r` y `connection received`: IP y puerto
¿Falló un acceso?	`FATAL: password authentication failed for user "app_web"`

No registra:

- **Las lecturas:** con `mod`, ningún `SELECT`. No se sabe quién leyó qué.
- **Lo que pasa dentro de funciones:** un `CALL sp_registrar_pedido(…)` no deja sus `INSERT`.
- **Las filas y los valores anteriores** de un `UPDATE`.
- **Si la transacción se confirmó:** un `INSERT` deshecho se ve igual que uno confirmado.
- **La aplicación real:** `app=` lo declara el cliente y se puede falsear.
- **Cuántos intentos hubo:** una conexión fallida dejó dos líneas `FATAL` (psql reintenta sin SSL).

pgaudit, a nivel de nociones: agrega auditoría por clase (`READ`, `WRITE`, `ROLE`, `DDL`) y por objeto. Podría registrar todo `SELECT` sobre `usuario` sin registrar el resto, y registra lo que se ejecuta dentro de funciones. Es lo que faltaría para responder «quién leyó la tabla de usuarios».

B.4 · ¿El log es un activo sensible?

Sí. Tiene mails, IP, el texto de las sentencias con los datos insertados y, como se vio en la Parte C, puede tener contraseñas si una sentencia las lleva como texto. Solo deberían leerlo el administrador de la base y quien audita: en el sistema operativo, el archivo pertenece a la cuenta del servicio de PostgreSQL (en Windows, permisos sobre `data\log` solo para esa cuenta y los administradores); en el motor, leerlo con `pg_read_file` exige ser superusuario o tener `pg_read_server_files`, y ningún rol de la aplicación lo tiene.

Parte C · Simulacro de incidente

Escenario: alguien obtuvo la contraseña de `app_web` (por ejemplo, de un archivo de configuración del servidor web) y se conecta directo a la base. `scripts/simulacro_incidente.sh` ejecuta `sql/simulacro_incidente.sql` como `app_web`, sin cortar ante errores.

#	Acción	Resultado
1	5 llamadas a `fn_autenticar` con contraseñas incorrectas para la misma cuenta de prueba	Rechazadas
2	`fn_autenticar` con la contraseña correcta	Inicio de sesión
3	`SELECT * FROM usuario`, `SELECT id, nombre, apellido, mail FROM usuario`, `SELECT * FROM vista_usuario_reportes`	Permiso denegado
4	`GRANT admin_datos TO rol_app_lectura`, `GRANT admin_datos TO app_web`, `SET ROLE admin_datos`, `fn_resetear_password(…)`	Permiso denegado

¿Falló el escalamiento?	Antes	Después
`app_web` es miembro de `admin_datos`	f	f
`rol_app_lectura` es miembro de `admin_datos`	f	f
Miembros de `admin_datos`	0	0
`auditoria_seguridad`: 5 `LOGIN` fallidos y 1 exitoso de `app_web`		

Sí, falló (`evidencias/C_simulacro_verificacion.txt`). Cada intento quedó en el log con su `ERROR`: nadie tiene `ADMIN OPTION` sobre `admin_datos`, y `fn_resetear_password` es exclusiva de soporte.

Hallazgo: en el log original, `fn_autenticar` dejaba la línea `STATEMENT` con la llamada completa, **contraseña incluida**. Se corrigió con `SET log_min_error_statement = 'panic'` en las dos funciones.

Log anonimizado (`docs/log_simulacro_anonimizado.txt`, con `scripts/anonimizar_log.py` y el mismo criterio que `usuario_anon`): mails → `anon_…@ejemplo.test`; IP y puerto → `ip_origen_N`; contraseñas en llamadas a funciones → `'***'`. El original no se sube al repositorio. Control: 0 apariciones de `@foodstore` en el archivo anonimizado.

C.4 y C.5 · Reconstrucción de la IA y contraste

OpenCode (agente Plan, en una carpeta fuera del repo que solo tenía el log anonimizado) reconstruyó el incidente sin otro contexto (`docs/respuestas_ia/C_reconstruccion_incidente.md`).

	Detalle
Qué acertó	El orden de los hechos: 5 intentos fallidos y 1 exitoso contra la misma cuenta; las tres lecturas de `usuario` y la vista rechazadas; los dos `GRANT`, el `SET ROLE` y `fn_resetear_password` rechazados. Concluyó bien que el escalamiento no tuvo éxito. Varias medidas coinciden con mi decisión: rotar la contraseña de `app_web`, limitar intentos y revisar las membresías.
Qué omitió	Lo más grave: la sesión de `app_web` ya estaba autenticada con su contraseña **antes** de los intentos: el atacante tenía la credencial de la aplicación. Tampoco señaló que, con `log_statement = mod`, un `SELECT` que sí funcionó (por ejemplo sobre `pedido`) no deja rastro.
Sin sustento en el log	(1) Que la fuerza bruta le dio el rol `app_web`, que ya tenía desde la conexión. (2) Bloquear `ip_origen_1` en el firewall: es un seudónimo de la anonimización y nada dice que el origen sea externo. (3) Terminar una sesión que el mismo log muestra cerrada a los 0,203 s. Lo detecté comparando cada afirmación con las líneas del log y con el SQL ejecutado.

C.6 · Decisión de contención (mía, no ejecutada automáticamente desde la IA)

Medida	Por qué	Evidencia
Cambiar la contraseña de `app_web` y sacarla del archivo de configuración del servidor web	La sesión se autenticó con la contraseña correcta	`conexión autenticada: identidad=«app_web»`
Restringir `app_web` en `pg_hba.conf` a la IP del servidor web	Hoy se acepta desde cualquier origen	Línea de `pg_hba.conf` de la conexión
Forzar el cambio de contraseña de la cuenta atacada	Su contraseña fue adivinada	5 fallos y 1 éxito sobre la misma cuenta

Medida	Por qué	Evidencia
Bloqueo temporal en `fn_autenticar` tras varios fallos seguidos	Hoy no hay límite de intentos	5 fallos en menos de 1 s
No revertir roles	El escalamiento falló y no hay membresías nuevas	`C_simulacro_verificacion.txt`
Durante la investigación, registrar las lecturas de `app_web` (`ALTER ROLE app_web SET log_statement = 'all'` o pgaudit)	Con `mod` no se sabe si leyó pedidos	Límite del registro (B.3)

Parte D · Anonimización y equivalencia

`sql/usuario_anon.sql` crea una copia determinista de `usuario`: mail → `anon_` + 10 caracteres de `sha256` + `@ejemplo.test`; nombre y apellido → `Nombre …` y `Apellido …` con otros caracteres del mismo hash; celular suprimido; contraseña no copiada; fecha de alta generalizada al mes (`mes_alta`); `rol` y `eliminado` se conservan porque son las dimensiones de los reportes; el `id` no se copia. Control: **0** mails, nombres o apellidos reales en la copia. *Limitación:* la sal es fija y está en el repositorio; con datos reales tendría que ser secreta.

Consulta: cantidad de usuarios por rol y por mes de alta, sobre `usuario_anon` y sobre `usuario` (`sql/equivalencia_agregados.sql`), comparadas con `EXCEPT` en los dos sentidos.

Resultado	Valor
Grupos (rol × mes)	17, todos coinciden
Grupos distintos	**0**
Usuarios en `usuario` / en `usuario_anon`	147 / 147

La consulta que escribió OpenCode a partir de la spec dio los mismos grupos que la de referencia: 0 diferencias en cada sentido (`evidencias/D_comprobacion_consultas_ia.txt`). Advirtió bien que, con `TIMESTAMPTZ`, el mes depende de la zona horaria de la sesión: por eso las dos se corren en la misma sesión.

D.4 · Qué no debería salir nunca hacia un asistente de IA sin anonimizar: los mails, nombres, apellidos y celulares de `usuario` y de `cliente`, y los hashes de `usuario.contrasena`. Tampoco las IP y los mails del log de auditoría, que están en `auditoria_seguridad` y en el log del servidor, ni un volcado de `pedido` unido a `cliente`, que liga a una persona con lo que compró. Para pedirle ayuda a la IA alcanza con la estructura de las tablas (`\d usuario`) o con `usuario_anon`: los agregados dan lo mismo.

Parte E · Bitácora y Git

`BITACORA.md` tiene la tabla de tres columnas (prompt enviado, respuesta relevante, validación o corrección humana) con un registro por cada uso de la IA. Resumen:

Uso de la IA	Validación o corrección
Claude · `roles.sql`	El `REVOKE EXECUTE` de los privilegios por defecto iba con `IN SCHEMA public` y no tenía efecto: se pasó a la forma global. **Error corregido antes de ejecutar.**
Claude · prueba de ventas	Con el `EXECUTE` revocado, `app_web` no podía vender: el trigger del resumen de la U4 llamaba a una función sin permiso. Las funciones de trigger pasaron a `SECURITY DEFINER`.
OpenCode · auditoría de roles	7 propuestas, comprobadas una por una con `ROLLBACK` (Parte A).
Claude · `fn_autenticar`	El log dejaba la contraseña en la línea `STATEMENT`. **Error detectado al probar.**
OpenCode · reconstrucción	Secuencia aceptada; tres afirmaciones sin sustento y una omisión grave (Parte C).
OpenCode · consulta equivalente	Aceptada: 0 diferencias.

Caso más significativo: la corrección que propuso OpenCode para que `admin_datos` no lea las contraseñas, `REVOKE SELECT (contrasena) ON usuario FROM admin_datos`. Leída parecía correcta, pero al probarla dentro de una transacción `has_column_privilege` siguió dando `t` y `admin_datos` leyó las 147 contraseñas igual: en

PostgreSQL, un permiso sobre la tabla entera cubre todas sus columnas. Si la hubiera aplicado sin probar, habría creído que el problema estaba resuelto.

Historial de Git: hay un commit por parte, de la A a la E. Pero las partes se hicieron y probaron en mi copia local y los commits se armaron al final, separando los archivos por parte: el historial muestra la división del trabajo, no el orden ni las correcciones del medio. Lo haría distinto: un commit al terminar cada parte y otro por cada corrección que salió de la revisión de la IA.

Declaración de uso de IA (DUIA)

Herramientas	**Claude** (Anthropic), asistente principal: escribió los scripts de referencia, los verificadores, el anonimizador del log, los borradores de los informes y este documento. **OpenCode** (agente Plan, en mi PC): auditó `roles.sql`, reconstruyó el incidente y escribió la consulta equivalente. **Kiro**: revisó la spec de roles.
Qué se aceptó y qué no	Detallado en las partes A, C, D y E, y en `BITACORA.md`. Lo que propuso la IA se probó en mi base dentro de `BEGIN … ROLLBACK` antes de decidir.
Verificación	Corrí todo en mi PC con `ejecutar_segunda_entrega.sh`; las salidas están en `evidencias/`. Nunca le pasé a una IA datos personales ni contraseñas: los usuarios son de prueba y el log que analizó OpenCode estaba anonimizado.

Anexo · Scripts principales

sql/roles.sql

```
-- U5 · roles.sql — Roles y permisos de Food Store con mínimo privilegio
--
-- Se ejecuta como postgres sobre la base del proyecto y se puede repetir: los roles se
-- crean si no existen y, antes de otorgar, se revoca todo lo anterior. Así, el resultado
-- es siempre exactamente lo que dice este archivo.
--
-- Las contraseñas NO se escriben acá. Después de correrlo, en psql:
--   \password app_web       \password admin_datos
--   \password usr_soporte   \password usr_reportes   \password usr_auditor
--
-- Justificación de cada rol (una oración cada uno):
--   rol_app_lectura   La tienda web muestra el catálogo y los pedidos: lee categorías, productos y pedidos, y nada con datos
personales.
--   rol_app_escritura La tienda web registra ventas: inserta pedidos y sus líneas y descuenta stock, sin poder cambiar
precios.
--   rol_soporte       La mesa de ayuda atiende a usuarios y clientes: los consulta sin ver contraseñas, corrige el celular o
la baja, y resetea contraseñas solo con la función.
--   rol_reportes      El área comercial mira resultados agregados: lee vistas de ventas y la tabla anonimizada de usuarios,
nunca datos personales.
--   rol_auditor       (adicional) Quien investiga un incidente lee los registros de auditoría, y ningún dato del negocio.
--   app_web           Login de la aplicación: lectura + escritura de la tienda y la función de autenticación.
--   admin_datos       Login del administrador de datos: todo sobre las tablas del esquema salvo modificar la auditoría, sin
ser superusuario ni poder crear roles.
--   usr_soporte, usr_reportes, usr_auditor  (adicionales) Logins personales de cada área, para que el log diga quién hizo
qué.

\set ON_ERROR_STOP on

-- 1. Crear los roles si no existen ------------------------------------------------
DO $$
DECLARE
    r TEXT;
BEGIN
    FOREACH r IN ARRAY ARRAY['rol_app_lectura', 'rol_app_escritura', 'rol_soporte', 'rol_reportes', 'rol_auditor'] LOOP
        IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = r) THEN
            EXECUTE format('CREATE ROLE %I NOLOGIN', r);
        END IF;
    END LOOP;
    FOREACH r IN ARRAY ARRAY['app_web', 'admin_datos', 'usr_soporte', 'usr_reportes', 'usr_auditor'] LOOP
        IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = r) THEN
            EXECUTE format('CREATE ROLE %I LOGIN', r);
        END IF;
    END LOOP;
END $$;

-- Ningún rol es superusuario ni puede crear roles o bases.
ALTER ROLE app_web      NOSUPERUSER NOCREATEDB NOCREATEROLE NOBYPASSRLS CONNECTION LIMIT 20;
ALTER ROLE admin_datos  NOSUPERUSER NOCREATEDB NOCREATEROLE NOBYPASSRLS CONNECTION LIMIT 3;
ALTER ROLE usr_soporte  NOSUPERUSER NOCREATEDB NOCREATEROLE NOBYPASSRLS CONNECTION LIMIT 5;
ALTER ROLE usr_reportes NOSUPERUSER NOCREATEDB NOCREATEROLE NOBYPASSRLS CONNECTION LIMIT 5;
ALTER ROLE usr_auditor  NOSUPERUSER NOCREATEDB NOCREATEROLE NOBYPASSRLS CONNECTION LIMIT 2;

-- 2. Punto de partida limpio -----------------------------------------------------
DO $$
DECLARE
    r TEXT;
BEGIN
    FOREACH r IN ARRAY ARRAY['rol_app_lectura', 'rol_app_escritura', 'rol_soporte', 'rol_reportes', 'rol_auditor',
                             'app_web', 'admin_datos', 'usr_soporte', 'usr_reportes', 'usr_auditor'] LOOP
        EXECUTE format('REVOKE ALL ON ALL TABLES    IN SCHEMA public FROM %I', r);
        EXECUTE format('REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM %I', r);
        EXECUTE format('REVOKE ALL ON ALL ROUTINES  IN SCHEMA public FROM %I', r);
        EXECUTE format('REVOKE ALL ON SCHEMA public FROM %I', r);
        EXECUTE format('REVOKE ALL ON DATABASE %I FROM %I', current_database(), r);
    END LOOP;
    -- Los permisos de columna no se van con REVOKE ALL ON ALL TABLES.
    EXECUTE 'REVOKE ALL (id, nombre, apellido, mail, celular, contrasena, rol, eliminado, created_at) ON usuario FROM
rol_soporte';
    EXECUTE 'REVOKE ALL (stock) ON producto FROM rol_app_escritura';
END $$;

-- PUBLIC (todos los roles) no se conecta, no usa el esquema y no ejecuta funciones nuevas.
DO $$ BEGIN EXECUTE format('REVOKE ALL ON DATABASE %I FROM PUBLIC', current_database()); END $$;
REVOKE ALL ON SCHEMA public FROM PUBLIC;
REVOKE EXECUTE ON ALL ROUTINES IN SCHEMA public FROM PUBLIC;

-- 3. Conexión y esquema ----------------------------------------------------------
DO $$
BEGIN
    EXECUTE format('GRANT CONNECT ON DATABASE %I TO app_web, admin_datos, usr_soporte, usr_reportes, usr_auditor',
                   current_database());
END $$;
GRANT USAGE ON SCHEMA public TO rol_app_lectura, rol_app_escritura, rol_soporte, rol_reportes, rol_auditor, admin_datos;

-- 4. Roles de grupo --------------------------------------------------------------

-- rol_app_lectura: catálogo y pedidos (sin cliente ni usuario)
GRANT SELECT ON categoria, producto, pedido, detalle_pedido, vista_productos_vigentes TO rol_app_lectura;
```

```
-- rol_app_escritura: registrar ventas. UPDATE solo sobre la columna stock: la app no cambia precios.
GRANT INSERT ON pedido, detalle_pedido TO rol_app_escritura;
GRANT UPDATE (stock) ON producto TO rol_app_escritura;
GRANT EXECUTE ON PROCEDURE sp_registrar_pedido(BIGINT, forma_pago_enum, JSONB) TO rol_app_escritura;
GRANT EXECUTE ON FUNCTION  fn_total_pedido(BIGINT)                             TO rol_app_escritura;

-- rol_soporte: permisos de COLUMNA sobre usuario. Ve todo menos la contraseña y solo
-- corrige celular y baja. Para la contraseña usa fn_resetear_password.
GRANT SELECT (id, nombre, apellido, mail, celular, rol, eliminado, created_at) ON usuario TO rol_soporte;
GRANT UPDATE (celular, eliminado) ON usuario TO rol_soporte;
GRANT SELECT ON cliente TO rol_soporte;
GRANT EXECUTE ON FUNCTION fn_resetear_password(TEXT, TEXT) TO rol_soporte;

-- rol_reportes: solo vistas y agregados. vista_usuario_reportes (U3) NO se otorga:
-- oculta la contraseña, pero expone mail y celular. Para usuarios, usa usuario_anon (Parte D).
GRANT SELECT ON mv_facturacion_cat_mes, v_top5_categorias, vista_productos_vigentes,
                vista_detalle_pedido_producto TO rol_reportes;

-- rol_auditor: registros de auditoría.
GRANT SELECT ON auditoria_seguridad, auditoria_precio TO rol_auditor;

-- 5. Logins ------------------------------------------------------------------------
GRANT rol_app_lectura, rol_app_escritura TO app_web;
GRANT EXECUTE ON FUNCTION fn_autenticar(TEXT, TEXT) TO app_web;     -- solo la app autentica

GRANT rol_soporte  TO usr_soporte;
GRANT rol_reportes TO usr_reportes;
GRANT rol_auditor  TO usr_auditor;

-- admin_datos: DML completo sobre los datos del esquema. No recibe TRUNCATE, REFERENCES ni
-- TRIGGER, no es dueño de los objetos y no tiene ADMIN OPTION sobre ningún rol.
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO admin_datos;
-- La auditoría es de solo lectura también para admin_datos (hallazgo #1 de la auditoría con
-- OpenCode, comprobado en evidencias/A_04): quien administra los datos no puede borrar ni
-- cambiar la evidencia de lo que hizo. Las filas las escriben fn_autenticar y el trigger
-- de precios, que son SECURITY DEFINER y corren como su dueño.
REVOKE INSERT, UPDATE, DELETE ON auditoria_seguridad, auditoria_precio FROM admin_datos;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO admin_datos;
GRANT EXECUTE ON ALL ROUTINES IN SCHEMA public TO admin_datos;

-- 6. Objetos futuros ---------------------------------------------------------------
-- Lo que postgres cree de acá en adelante hereda la misma política: nadie recibe acceso
-- automático salvo admin_datos, y las funciones nuevas no quedan abiertas a PUBLIC.
-- El REVOKE va sin IN SCHEMA: los privilegios por esquema solo se SUMAN a los globales,
-- y el EXECUTE a PUBLIC es global. Con IN SCHEMA, el REVOKE no tendría efecto.
ALTER DEFAULT PRIVILEGES FOR ROLE postgres
    REVOKE EXECUTE ON ROUTINES FROM PUBLIC;
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO admin_datos;
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public
    GRANT USAGE, SELECT ON SEQUENCES TO admin_datos;
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public
    GRANT EXECUTE ON ROUTINES TO admin_datos;
```

sql/configurar_auditoria.sql

```
-- U5 · Parte B — Activación de la auditoría nativa del servidor
-- Se ejecuta como postgres. ALTER SYSTEM escribe en postgresql.auto.conf y no se puede
-- correr dentro de una transacción.

\echo '== Valores ANTES del cambio'
SELECT name, setting, context FROM pg_settings
 WHERE name IN ('logging_collector', 'log_destination', 'log_directory', 'log_connections',
                'log_disconnections', 'log_statement', 'log_line_prefix', 'log_min_error_statement',
                'log_file_mode')
 ORDER BY name;

ALTER SYSTEM SET log_connections         = 'on';      -- quién se conecta, desde dónde y si se autentica
ALTER SYSTEM SET log_disconnections      = 'on';      -- cuándo termina cada sesión y cuánto duró
ALTER SYSTEM SET log_statement           = 'mod';     -- DDL e INSERT/UPDATE/DELETE/TRUNCATE/COPY FROM
ALTER SYSTEM SET log_min_error_statement = 'error';   -- texto de toda sentencia que falla
-- Prefijo: hora con milisegundos, PID, usuario@base, origen (IP y puerto) y aplicación.
ALTER SYSTEM SET log_line_prefix         = '%m [%p] %q%u@%d %r app=%a ';

SELECT pg_reload_conf();
SELECT pg_sleep(1);

-- log_connections y log_disconnections rigen para las sesiones nuevas: se abre una.
\c
\echo '== Valores DESPUÉS del cambio (en una sesión nueva)'
SELECT name, setting FROM pg_settings
 WHERE name IN ('logging_collector', 'log_connections', 'log_disconnections', 'log_statement',
                'log_line_prefix', 'log_min_error_statement')
 ORDER BY name;

\echo '== Archivo de log actual (si sale vacío, logging_collector está apagado: ver docs/informe_auditoria.md)'
SELECT pg_current_logfile() AS archivo_de_log;
```

sql/auditoria_test.sql

```
-- U5 · Parte B — Secuencia de prueba para la auditoría
-- Se ejecuta conectado como app_web (lo hace scripts/correr_auditoria_test.sh).
-- El intento de inicio de sesión fallido lo provoca el script de shell, porque ocurre
```

```
-- antes de que exista una sesión.

\echo '-- 1. Lectura: catálogo vigente'
SELECT id_producto, nombre, precio FROM vista_productos_vigentes ORDER BY id_producto LIMIT 3;

\echo '-- 2. Inserción: un pedido de prueba con una línea'
INSERT INTO pedido (forma_pago, id_cliente) VALUES ('EFECTIVO', 1) RETURNING id_pedido AS pedido_prueba \gset
INSERT INTO detalle_pedido (cantidad, precio_unitario, id_pedido, id_producto)
SELECT 1, precio, :pedido_prueba, id_producto FROM producto WHERE nombre = 'Agua mineral 2L';

\echo '-- 3. Actualización: se descuenta el stock vendido'
UPDATE producto SET stock = stock - 1 WHERE nombre = 'Agua mineral 2L';

SELECT current_user AS conectado_como, :pedido_prueba AS pedido_creado;
```

`sql/simulacro_incidente.sql` (las cuentas y contraseñas son de prueba, inventadas)

```
-- U5 · Parte C — Simulacro de incidente de acceso indebido (SOLO sobre la base propia de desarrollo)
--
-- Escenario: alguien obtuvo la contraseña de app_web (por ejemplo, de un archivo de
-- configuración del servidor web) y se conecta directamente a la base.
-- Lo ejecuta scripts/simulacro_incidente.sh, conectado como app_web.
-- No se corta ante errores: los rechazos son parte de lo que se quiere registrar.

\set ON_ERROR_STOP off

\echo '-- 1. Intentos fallidos de inicio de sesión contra fn_autenticar (fuerza bruta sobre una cuenta)'
SELECT * FROM fn_autenticar('usuario007@foodstore.test', '123456');
SELECT * FROM fn_autenticar('usuario007@foodstore.test', 'password');
SELECT * FROM fn_autenticar('usuario007@foodstore.test', 'foodstore');
SELECT * FROM fn_autenticar('usuario007@foodstore.test', 'Prueba-000');
SELECT * FROM fn_autenticar('usuario007@foodstore.test', 'Prueba-006');

\echo '-- 2. Inicio de sesión exitoso'
SELECT * FROM fn_autenticar('usuario007@foodstore.test', 'Prueba-007');

\echo '-- 3. Lectura masiva de la tabla usuario'
SELECT * FROM usuario;
SELECT id, nombre, apellido, mail FROM usuario;
SELECT * FROM vista_usuario_reportes;

\echo '-- 4. Intento de escalamiento de privilegios'
GRANT admin_datos TO rol_app_lectura;
GRANT admin_datos TO app_web;
SET ROLE admin_datos;
SELECT fn_resetear_password('usuario007@foodstore.test', 'Tomada-por-el-atacante');

\echo '-- Fin de la secuencia'
SELECT current_user AS sigue_siendo, session_user AS sesion;
```

`sql/usuario_anon.sql`

```
-- U5 · Parte D — usuario_anon: copia anonimizada y determinista de usuario
--
-- Criterio (el mismo que aplica scripts/anonimizar_log.py al log):
--   mail        -> seudónimo: 'anon_' + 10 primeros caracteres de sha256('foodstore-u5:' || mail) + '@ejemplo.test'
--   nombre      -> 'Nombre '   + 6 caracteres del mismo hash
--   apellido    -> 'Apellido ' + otros 6 caracteres del hash
--   celular     -> se suprime (NULL)
--   contrasena  -> no se copia
--   created_at  -> se generaliza al mes: columna mes_alta (DATE, primer día del mes)
--   rol, eliminado -> se conservan: son las dimensiones de los reportes
--   id          -> no se copia: usuario_anon tiene su propio id, sin vínculo con usuario
-- Es determinista: el mismo mail da siempre el mismo seudónimo, así que reconstruirla da la misma tabla.
-- Limitación: la sal es fija y está en el repositorio. Con datos reales, la sal tendría que ser
-- secreta; si no, quien tenga una lista de mails podría recalcular los seudónimos.

DROP TABLE IF EXISTS usuario_anon;

CREATE TABLE usuario_anon (
    id_anon     BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre      TEXT        NOT NULL,
    apellido    TEXT        NOT NULL,
    mail        TEXT        NOT NULL UNIQUE,
    rol         rol         NOT NULL,
    eliminado   BOOLEAN     NOT NULL,
    mes_alta    DATE        NOT NULL CHECK (mes_alta = date_trunc('month', mes_alta)::DATE)
);

INSERT INTO usuario_anon (nombre, apellido, mail, rol, eliminado, mes_alta)
SELECT 'Nombre '   || substr(h, 11, 6),
       'Apellido ' || substr(h, 17, 6),
       'anon_' || substr(h, 1, 10) || '@ejemplo.test',
       rol, eliminado,
       date_trunc('month', created_at)::DATE
  FROM (SELECT u.*, encode(sha256(convert_to('foodstore-u5:' || lower(u.mail), 'UTF8')), 'hex') AS h
          FROM usuario u) x
 ORDER BY h;

-- Los reportes leen esta tabla, no la real.
GRANT SELECT ON usuario_anon TO rol_reportes;

\echo '== Muestra de usuario_anon'
SELECT nombre, apellido, mail, rol, eliminado, mes_alta FROM usuario_anon ORDER BY mail LIMIT 5;
```

```
\echo '== Controles: ningún mail, nombre ni celular real quedó en la copia (se espera 0)'
SELECT COUNT(*) AS datos_reales_filtrados
  FROM usuario_anon a
 WHERE a.mail IN (SELECT mail FROM usuario)
    OR a.nombre IN (SELECT nombre FROM usuario)
    OR a.apellido IN (SELECT apellido FROM usuario);
```

sql/equivalencia_agregados.sql

```
-- U5 · Parte D — Verificación de equivalencia de los agregados
-- Consulta 1 (sobre usuario_anon) y consulta 2 (la equivalente sobre usuario):
-- cantidad de usuarios por rol y por mes de alta.
-- Las dos se corren en la misma sesión: el mes de un TIMESTAMPTZ depende de la zona horaria.

\echo '== Consulta 1: usuario_anon'
SELECT rol, to_char(mes_alta, 'YYYY-MM') AS mes_alta, COUNT(*) AS usuarios
  FROM usuario_anon
 GROUP BY 1, 2
 ORDER BY 1, 2;

\echo '== Consulta 2: usuario (tabla real)'
SELECT rol, to_char(date_trunc('month', created_at), 'YYYY-MM') AS mes_alta, COUNT(*) AS usuarios
  FROM usuario
 GROUP BY 1, 2
 ORDER BY 1, 2;

\echo '== Lado a lado'
WITH a AS (SELECT rol, to_char(mes_alta, 'YYYY-MM') AS mes_alta, COUNT(*) AS n
             FROM usuario_anon GROUP BY 1, 2),
     r AS (SELECT rol, to_char(date_trunc('month', created_at), 'YYYY-MM') AS mes_alta, COUNT(*) AS n
             FROM usuario GROUP BY 1, 2)
SELECT COALESCE(r.rol, a.rol) AS rol, COALESCE(r.mes_alta, a.mes_alta) AS mes_alta,
       r.n AS usuarios_real, a.n AS usuarios_anon,
       CASE WHEN r.n IS NOT DISTINCT FROM a.n THEN 'sí' ELSE 'NO' END AS coincide
  FROM r FULL JOIN a ON a.rol = r.rol AND a.mes_alta = r.mes_alta
 ORDER BY 1, 2;

\echo '== Resumen (se espera: 0 grupos distintos y el mismo total)'
WITH a AS (SELECT rol, mes_alta AS mes, COUNT(*) AS n FROM usuario_anon GROUP BY 1, 2),
     r AS (SELECT rol, date_trunc('month', created_at)::DATE AS mes, COUNT(*) AS n FROM usuario GROUP BY 1, 2)
SELECT (SELECT COUNT(*) FROM ((SELECT * FROM r EXCEPT SELECT * FROM a)
                              UNION ALL (SELECT * FROM a EXCEPT SELECT * FROM r)) d) AS grupos_distintos,
       (SELECT COUNT(*) FROM usuario)      AS total_real,
       (SELECT COUNT(*) FROM usuario_anon) AS total_anon;
```